



LÊ NHẬT HUY
PHÙNG THANH TRIỀU
TRẦN HẠO KHIÊM
DƯƠNG MINH THÔNG
NGÔ GIA BẢO

Giới thiệu bài Lab

Bài Lab hướng dẫn thiết kế hệ thống Zynq SoC cơ bản bằng Vivado IP Integrator và SDK/Vitis trên nền tảng ZedBoard/PynQ-Z2. Quy trình thực hiện HW/SW Codesign gồm tạo Block Design, kết nối AXI GPIO, generate bitstream và viết chương trình C điều khiển LED trên Zynq FPGA.

Mục tiêu bài Lab

- Hiểu kiến trúc và quy trình thiết kế Hardware/Software Codesign trên nền tảng Zynq SoC.
- Làm quen với Vivado IDE, IP Integrator, AXI interface và quá trình tạo Block Design.
- Thực hiện generate bitstream, export hardware và phát triển ứng dụng C chạy trên ARM Cortex-A9.
- Điều khiển phần cứng trong Programmable Logic (PL) thông qua software trên Processing System (PS).

Công cụ cần thiết

Phần cứng

- ZedBoard hoặc PynQ-Z2
- USB JTAG/UART Cable

Phần mềm

- Vivado Design Suite HLx 2019.2
- Vivado SDK / Vitis

Kết quả đạt được

- Xây dựng thành công hệ thống Zynq SoC cơ bản.
- Kết nối AXI GPIO với ARM Cortex-A9 Processor.
- Generate và nạp bitstream xuống FPGA.
- Chạy chương trình điều khiển LED trên Zynq.
- Hiểu quy trình Hardware/Software Codesign cơ bản trên FPGA SoC.

Phần công việc thực hiện

A. Câu hỏi

1 Giải thích sự khác nhau giữa Processing System (PS) và Programmable Logic (PL).

Processing System (PS) là phần "phần cứng cố định" của chip — một bộ vi xử lý đã được thiết kế và sản xuất sẵn. Trên Zynq-7000, đây là lõi ARM Cortex-A9 (dual-core); trên Zynq UltraScale+ là Cortex-A53 và Cortex-R5. PS hoạt động giống như một con vi xử lý ARM thông thường: chạy phần mềm tuần tự (firmware, RTOS, hoặc Linux), có sẵn các ngoại vi như UART, SPI, I2C, Ethernet, USB, bộ điều khiển DDR memory, v.v.

Programmable Logic (PL) chính là phần FPGA — một mảng các khối logic có thể cấu hình lại được (configurable logic blocks, LUTs, flip-flops, DSP slices, Block RAM). PL không có "chức năng" cố định khi xuất xưởng; bạn nạp một file bitstream để định nghĩa nó sẽ trở thành mạch số gì.

Khía cạnh	Processing System (PS)	Programmable Logic (PL)
Bản chất	Vi xử lý ARM cố định (hard processor)	Mảng logic lập trình được (FPGA fabric)
Cách "lập trình"	Viết phần mềm (C/C++, Linux...)	Thiết kế mạch số bằng HDL (VHDL/Verilog)
Mô hình xử lý	Tuần tự, theo lệnh	Song song, theo phần cứng
Điểm mạnh	Điều khiển luồng, logic phức tạp, hệ điều hành	Xử lý song song tốc độ cao, độ trễ thấp, deterministic
Cấu hình lại	Chạy chương trình khác = nạp phần mềm	Đổi chức năng = nạp bitstream mới
Tiêu thụ tài nguyên	Có sẵn, không tốn tài nguyên FPGA	Mỗi chức năng tốn LUT/FF/BRAM

2 Vai trò của AXI Interconnect trong hệ thống Zynq là gì?

AXI Interconnect là khối "trung gian định tuyến" — nó kết nối nhiều **Master** (thiết bị khởi tạo giao dịch) với nhiều **Slave** (thiết bị đáp ứng giao dịch) trên cùng một hệ thống bus AXI. Nói ngắn gọn, nó đóng vai trò như một "tổng đài" hoặc "ngã tư có điều khiển" cho dữ liệu di chuyển giữa PS và các khối IP trong PL.

Trong một thiết kế Zynq điển hình, bạn thường có một Master (ví dụ cổng AXI từ PS) cần truy cập nhiều Slave khác nhau trong PL: một khối GPIO, một bộ điều khiển DMA, một khối tăng tốc xử lý ảnh, một BRAM controller... Mỗi khối nằm ở một dải địa chỉ riêng. AXI Interconnect đứng giữa, nhận giao dịch từ Master rồi chuyển đến đúng Slave dựa trên địa chỉ. Nếu không có nó, bạn không thể nối một Master vào nhiều Slave một cách trực tiếp.

Các chức năng chính

- Định tuyến theo địa chỉ (address decoding) — Đây là nhiệm vụ cốt lõi. Mỗi Slave được gán một dải địa chỉ trong Address Editor của Vivado. Khi Master phát ra giao dịch tới một địa chỉ, Interconnect giải mã địa chỉ đó và chuyển giao dịch đến đúng Slave tương ứng.
- Phân xử truy cập (arbitration) — Khi nhiều Master cùng muốn truy cập một Slave, Interconnect quyết định ai được phục vụ trước, đảm bảo không xung đột và tránh việc một Master bị "đói" tài nguyên.
- Chuyển đổi giao thức (protocol conversion) — Interconnect có thể kết nối các thành phần dùng biến thể AXI khác nhau, ví dụ một Master AXI4 với một Slave AXI4-Lite, hoặc xử lý giao dịch giữa AXI3 (của PS Zynq-7000) và AXI4.
- Chuyển đổi độ rộng dữ liệu (data width conversion) — Nếu Master có bus dữ liệu 64-bit còn Slave chỉ 32-bit, Interconnect tự động xử lý việc đóng gói/tách dữ liệu cho khớp.
- Chuyển đổi miền/tần số clock (clock domain crossing) — PS và các khối trong PL có thể chạy ở tần số khác nhau. Interconnect chứa các FIFO bất đồng bộ để dữ liệu vượt qua ranh giới clock một cách an toàn.
- Đệm dữ liệu (buffering / register slices) — Có thể chèn các tầng thanh ghi để cải thiện timing closure, đặc biệt khi đường dây dài hoặc tần số cao.

3 Tại sao cần Processor System Reset IP?

Ta cần **Processor System Reset IP** vì khối này **cung cấp các tín hiệu reset đã được tùy biến (customized resets)** cho toàn bộ hệ thống xử lý, bao gồm cả 3 đối tượng:

- **Bộ vi xử lý (processor)** — tức Zynq PS.
- **Khối AXI Interconnect** — bộ kết nối AXI giữa PS và các IP trong PL.
- **Các thiết bị ngoại vi (peripherals)** — ví dụ trong bài tutorial này là khối **AXI GPIO** điều khiển LED.

Nói cách khác, khi bạn thêm bất kỳ một IP ngoại vi nào (như AXI GPIO) và kết nối nó với Zynq PS qua bus AXI, Vivado **tự động chèn thêm khối Processor System Reset Module** vì hệ thống cần một khối chuyên trách để **sinh ra và phân phối các tín hiệu reset phù hợp** cho từng thành phần đó.

4 AXI GPIO hoạt động như thế nào trong bài Lab này?

AXI GPIO là một **bộ điều khiển GPIO được triển khai trong PL** (Programmable Logic) của Zynq, đóng vai trò làm **cầu nối giữa Zynq PS (phần xử lý ARM) và 4 LED trên ZedBoard**

Mục đích: cho phép phần mềm chạy trên ARM **điều khiển LED phần cứng** thông qua việc đọc/ghi vào các thanh ghi của AXI GPIO.

Chuỗi hoạt động của AXI GPIO:

- **Tại PL:** Khối AXI GPIO được tổng hợp vào FPGA, một mặt nối với AXI Interconnect (giao tiếp với PS), mặt khác nối ra 8 chân LED của ZedBoard.
- **Khi nạp bitstream + chạy chương trình:** ARM thực thi hàm XGpio_Initialize() → driver biết được địa chỉ 0x41200000 của AXI GPIO nhờ xparameters.h.
- **Cấu hình hướng I/O:** XGpio_SetDataDirection() ghi vào thanh ghi điều hướng để đặt 8 bit là output.
- **Điều khiển LED:** ARM ghi dữ liệu (ví dụ luân phiên hai mẫu bit) qua bus AXI → AXI Interconnect định tuyến tới AXI GPIO → AXI GPIO xuất ra 8 chân vật lý → **LED sáng/tắt tương ứng**.

5 Trình bày quy trình từ Block Design đến Generate Bitstream.

Quy trình từ lúc thiết kế khối (Block Design) cho đến khi tạo tệp Bitstream (Generate Bitstream) trên phần mềm Vivado bao gồm các bước sau:

a. Tạo và cấu hình Block Design

- Khởi tạo thiết kế bằng cách chọn **Create Block Design** trong cửa sổ Flow Navigator.
- Thêm các khối IP (Add IP) cần thiết vào bản vẽ (Diagram canvas).
- Sử dụng các công cụ hỗ trợ như **Run Block Automation** để tự động cấu hình các giao tiếp bên ngoài (ví dụ: DDR, FIXED_IO) và **Run Connection Automation** để tự động kết nối các khối IP với nhau thông qua chuẩn giao tiếp (ví dụ: AXI).

b. Lưu và Kiểm tra thiết kế (Save & Validate Design)

- Lưu thiết kế khối bằng cách chọn **File > Save Block Design**.
- Xác thực thiết kế bằng cách chọn **Tools > Validate Design** trên Menu Bar hoặc nhấn nút biểu tượng Validate Design trên thanh công cụ. Quá trình này sẽ chạy kiểm tra luật thiết kế (Design-Rule-Check - DRC).
- Một hộp thoại thông báo sẽ hiện ra để xác nhận thiết kế đã được kiểm tra thành công, nhấn OK để đóng.

c. Tạo tệp bọc HDL (Create HDL Wrapper)

- Chuyển sang cửa sổ **Sources**, nhấn chuột phải vào thiết kế hệ thống ở cấp cao nhất (top-level system design) và chọn **Create HDL Wrapper**.

- Khi hộp thoại Create HDL Wrapper mở ra, hãy chọn **Let Vivado manage wrapper and auto-update** rồi nhấn OK. Quá trình này sẽ tạo ra tệp mã nguồn HDL (VHDL hoặc Verilog) bao bọc hệ thống của bạn.

d. Tổng hợp, Thực thi và Tạo Bitstream (Synthesis, Implementation & Generate Bitstream)

- Sau khi tệp HDL được tạo, tìm đến mục *Program and Debug* trong cửa sổ Flow Navigator và nhấn chọn **Generate Bitstream**.

- Nếu có một hộp thoại hiện ra yêu cầu lưu thiết kế, hãy nhấn **Save**.

- Tiếp theo, một hộp thoại sẽ yêu cầu bạn phải chạy quá trình Tổng hợp (Synthesis) và Thực thi (Implementation) trước khi bắt đầu tạo Bitstream. Nhấn **Yes** để đồng ý.

- Vivado sẽ chạy kết hợp ba quá trình: Synthesis, Implementation, và Generate Bitstream liên tục. Quá trình này có thể mất vài phút tùy thuộc vào cấu hình máy tính.

- Khi quá trình hoàn tất, hộp thoại *Bitstream Generation Completed* sẽ hiển thị báo hiệu việc tạo tệp Bitstream đã thành công. Bạn có thể chọn **Open Implemented Design** và nhấn OK để xem tài nguyên thiết kế đã được thực thi.

6 Cách kết nối board Zynq với máy tính

Không giống như các thiết bị FPGA của Xilinx, các thiết bị AP SoC như Zynq-7010 được thiết kế xoay quanh bộ xử lý, bộ xử lý này đóng vai trò là bộ điều khiển chính (master) đối với phần logic có thể lập trình được và tất cả các ngoại vi trên chip trong hệ thống xử lý.

Điều này khiến quá trình khởi động của Zynq giống với vi điều khiển hơn là FPGA.

Quá trình khởi động gồm 3 bước:

+ Bước 1: Sau khi bo mạch ZYBO được cấp nguồn hoặc khi Zynq được reset bằng phần mềm hoặc bằng cách nhấn nút PS-SRST, một trong các bộ xử lý sẽ bắt đầu thực thi một đoạn mã chỉ đọc nội bộ gọi là BootROM. Nếu và chỉ nếu Zynq vừa mới được cấp nguồn, BootROM trước tiên sẽ chốt trạng thái của các chân mode vào thanh ghi mode, các chân mode này được nối với JP5 trên ZYBO. Nếu BootROM được thực thi do reset, thì các chân mode sẽ không được chốt lại và trạng thái trước đó của thanh ghi mode sẽ được sử dụng.

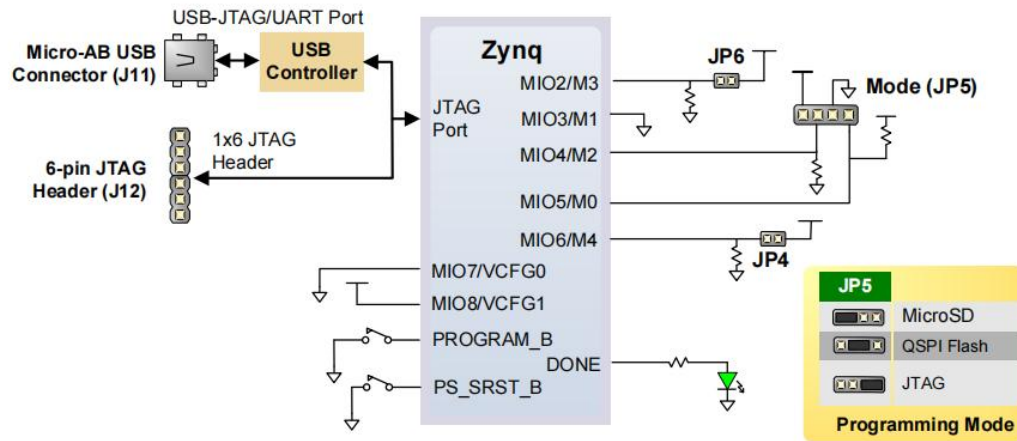
Điều này có nghĩa là ZYBO cần phải tắt rồi bật nguồn lại để ghi nhận bất kỳ thay đổi nào ở jumper chế độ lập trình JP5. Tiếp theo, BootROM sẽ sao chép một FSBL (First Stage Bootloader) từ bộ nhớ non-volatile memory được chỉ định bởi thanh ghi mode vào vùng RAM nội bộ 256 KB bên trong APU. FSBL phải được đóng gói bên trong một Zynq Boot Image để BootROM có thể sao chép nó đúng cách. Công việc cuối cùng của BootROM là chuyển quyền thực thi chương trình cho FSBL nằm trong OCM.

+ Bước 2: Ở bước này, FSBL trước tiên sẽ hoàn tất việc cấu hình các thành phần của Processing System, chẳng hạn như bộ điều khiển bộ nhớ DDR. Sau đó, nếu trong Zynq Boot Image có chứa bitstream, bitstream này sẽ được đọc và dùng để cấu hình phần Programmable Logic. Cuối cùng, user application sẽ được nạp vào bộ nhớ từ Zynq Boot Image, và quyền thực thi chương trình sẽ được chuyển giao cho ứng dụng đó.

+ Bước 3: Bước cuối cùng là thực thi user application đã được FSBL nạp vào bộ nhớ. Ứng dụng này có thể là bất kỳ loại chương trình nào, từ một thiết kế đơn giản như “Hello World” cho đến một Second Stage Bootloader dùng để khởi động hệ điều hành như Linux.

ZYBO hỗ trợ ba chế độ khởi động (boot mode) khác nhau: microSD, QSPI Flash và JTAG. Chế độ khởi

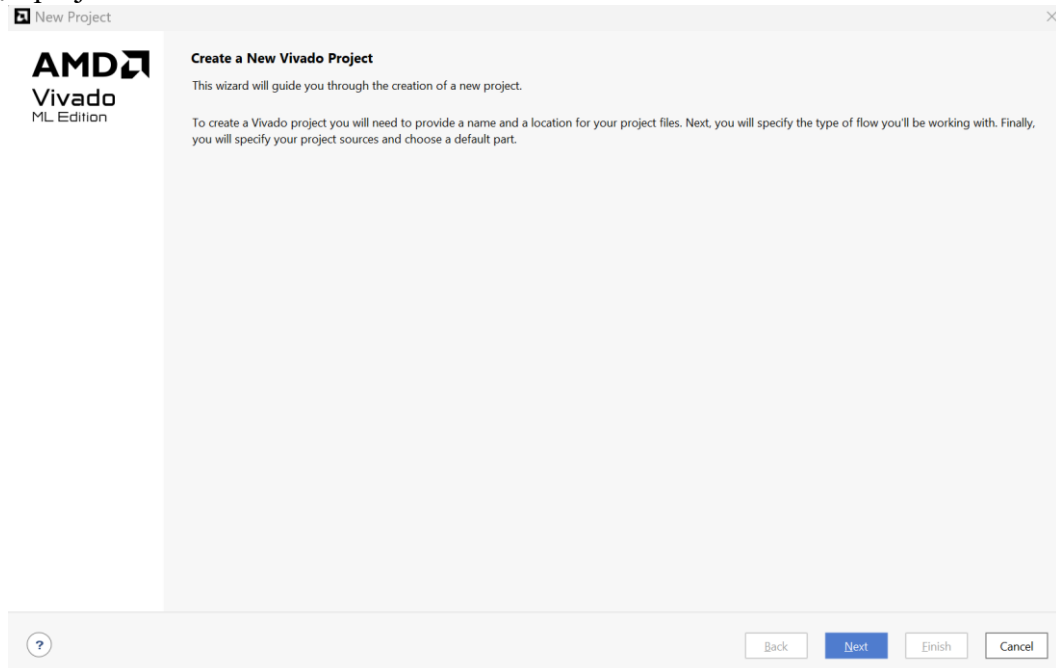
động được lựa chọn bằng jumper Mode (JP5), jumper này sẽ ảnh hưởng đến trạng thái của các chân cầu hình của Zynq sau khi cấp nguồn.



Hình 0: Zybo configuration pins

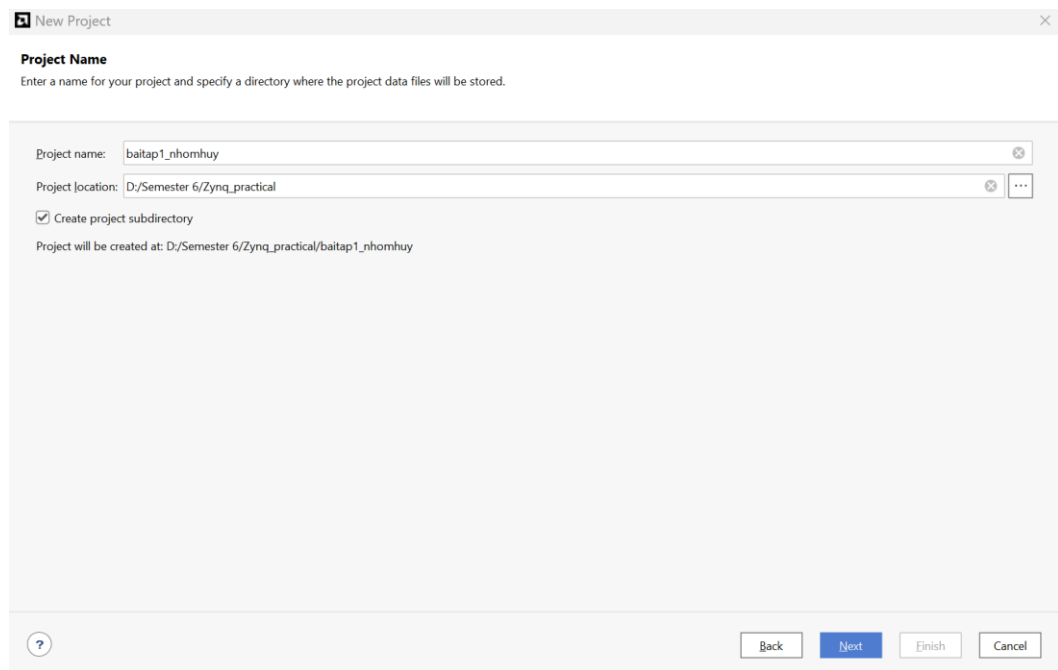
B. Thực hành

- Tạo project mới



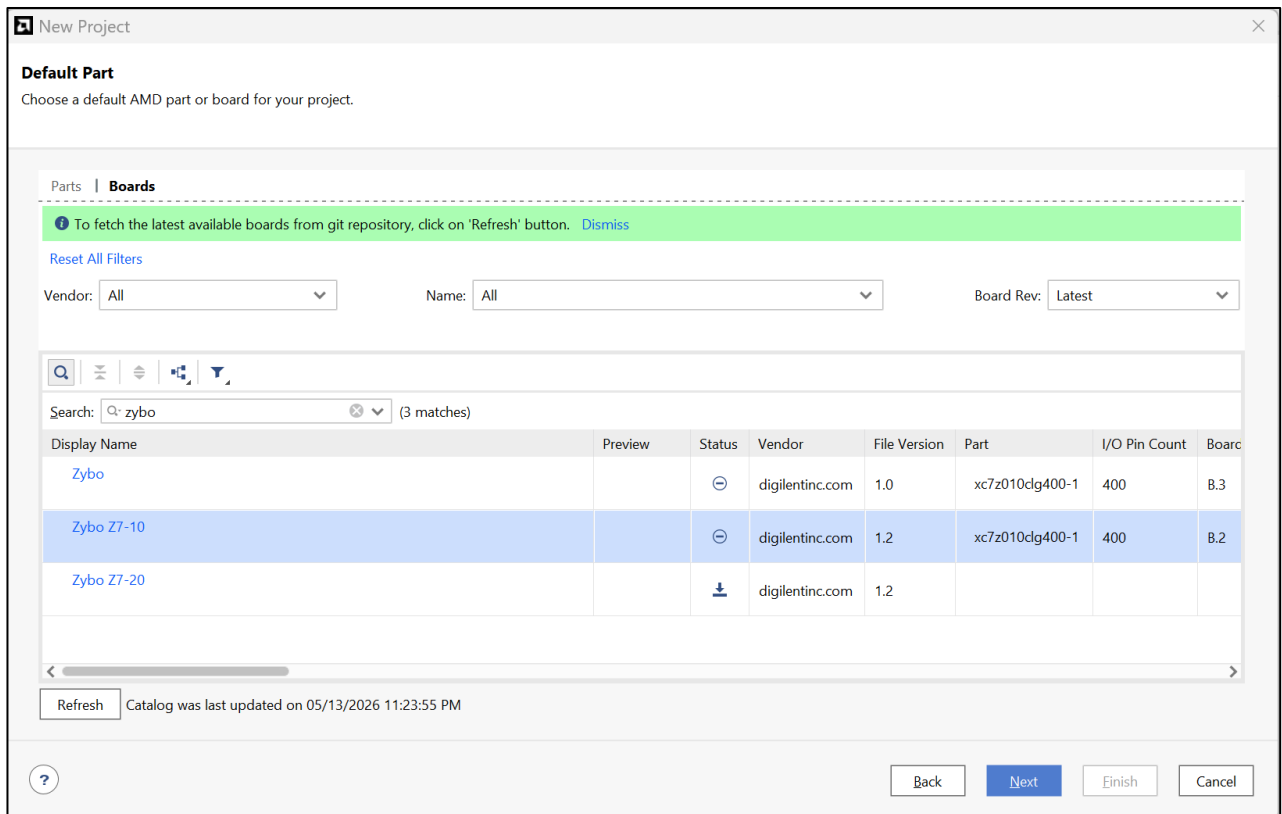
Hình 1: Giao diện tạo project

- Đặt tên và chọn địa chỉ lưu cho project



Hình 2: Đặt tên và đường dẫn

- **Add Sources:**
 - + Target language: **Verilog**
 - + Simulator language: **Mixed**
 - + Click **Next** (không thêm file nguồn).
- **Add Existing IP** → **Next** (bỏ qua).
- **Add Constraints** → **Next** (bỏ qua).
- **Default Part:**
 - + Chọn tab **Boards**
 - + Display Name: chọn **Zybo Z7 - 10**
 - + Board Rev: chọn rev đúng với board của bạn (vd B.3)
 - + **Next**



Hình 3: Chọn Board Zybo Z7 - 10

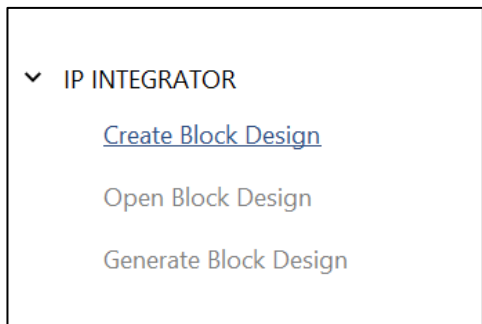
- **Summary → Finish.**

Vivado sẽ mở giao diện chính với Flow Navigator bên trái, Workspace ở giữa, và Results window phía dưới

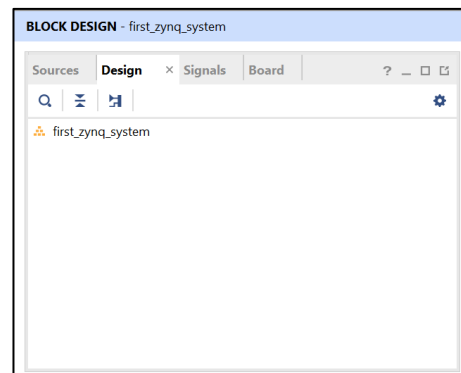
- **Tạo Block Design**

(a) Trong Flow Navigator → **IP Integrator** → **Create Block Design**.

(b) Design name: `first_zynq_system` → **OK**. Canvas Diagram sẽ mở ra.

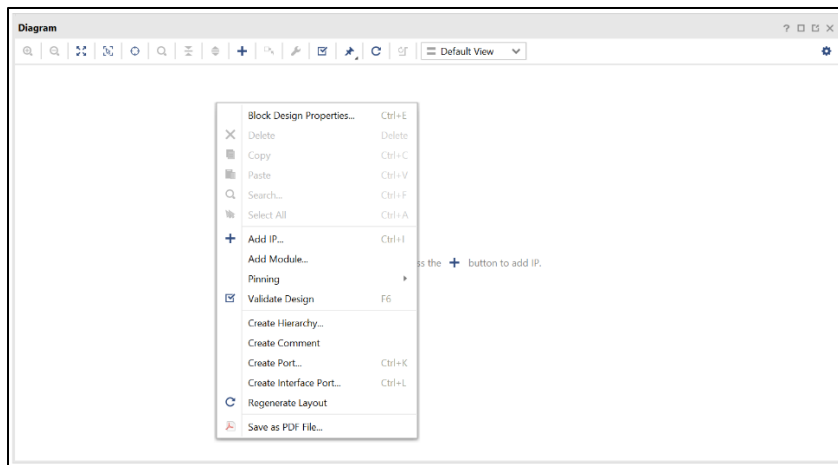


Hình 4: Tạo block design

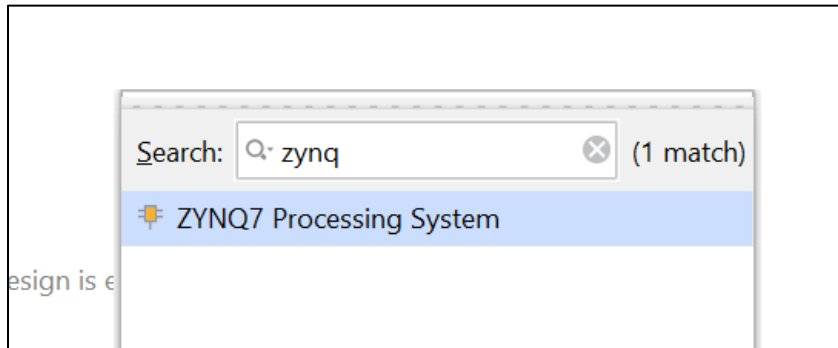


Hình 5: Sau khi tạo block

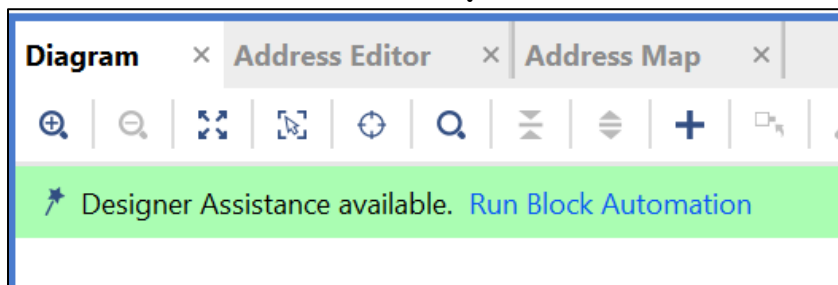
- Right-click trong canvas → **Add IP** (hoặc nút "+" ở toolbar trái).
- Tìm zynq, double-click **ZYNQ7 Processing System**
- Một thanh xanh "Designer Assistance" xuất hiện phía trên → click **Run Block Automation**:
- + Tick **Apply Board Preset** → **OK**
- + Vivado sẽ tự tạo các interface DDR và FIXED_IO ra ngoài top-level.



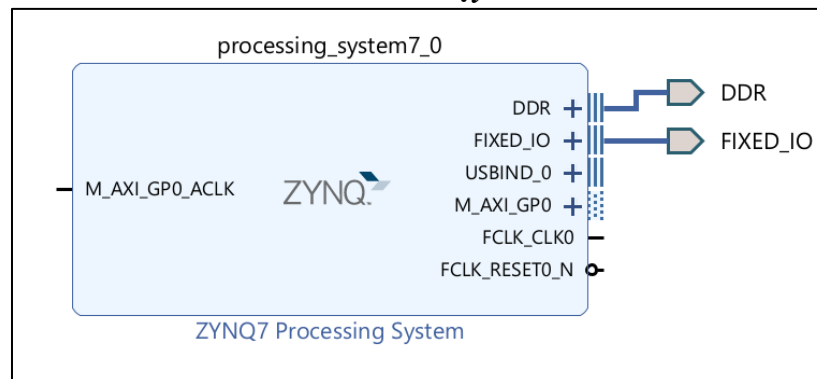
Hình 6: Thêm IP



Hình 6: Chọn PS

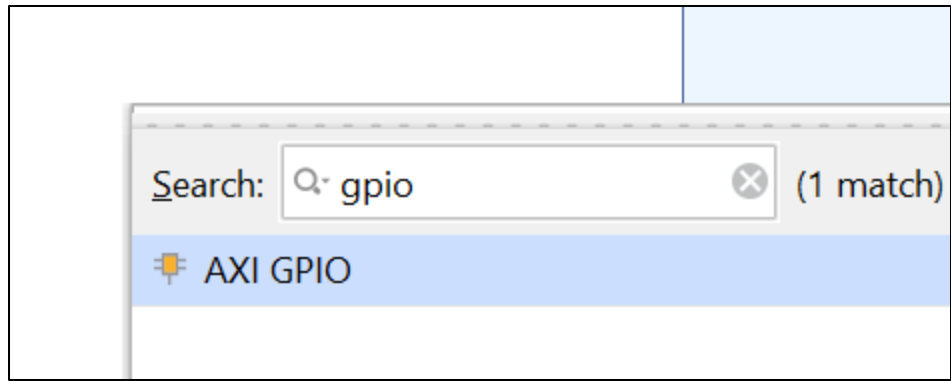


Hình 7: Khởi chạy khối PS

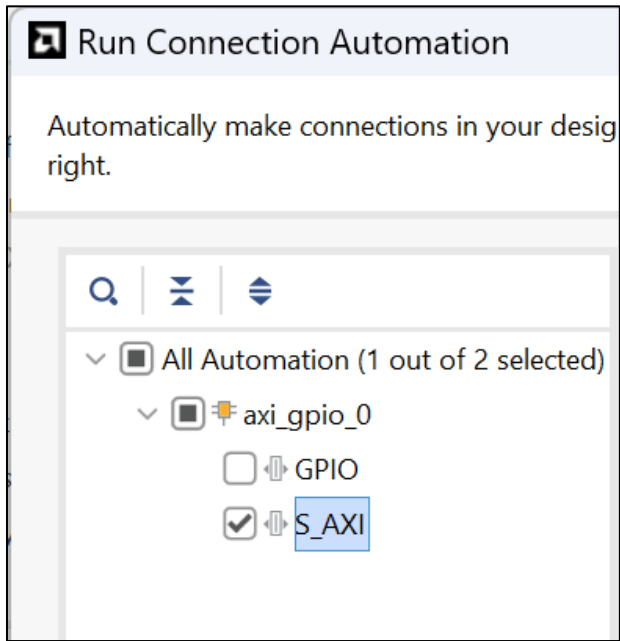


Hình 8: Sau khi khởi chạy khối PS

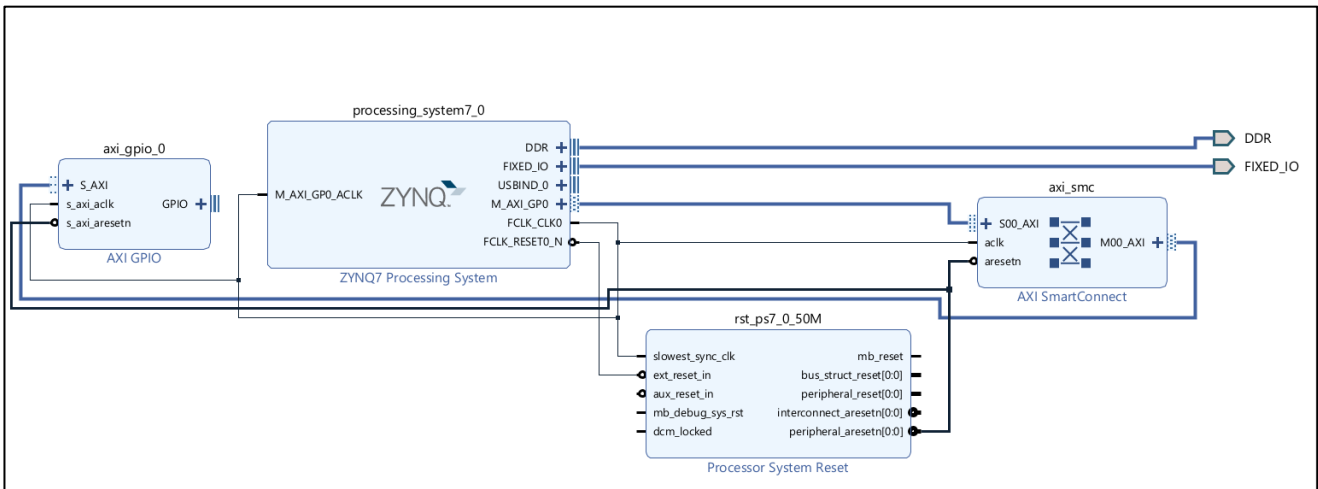
- Right-click → **Add IP** → gõ GPIO → thêm **AXI GPIO**.
 - Click **Run Connection Automation**:
- + Chọn /axi_gpio_0/S_AXI → **OK**
 - + Vivado sẽ tự thêm **Processor System Reset** và **AXI Interconnect**, đồng thời nối tự động giữa PS ↔ AXI Interconnect ↔ AXI GPIO.



Hình 9: Thêm IP của GPIO

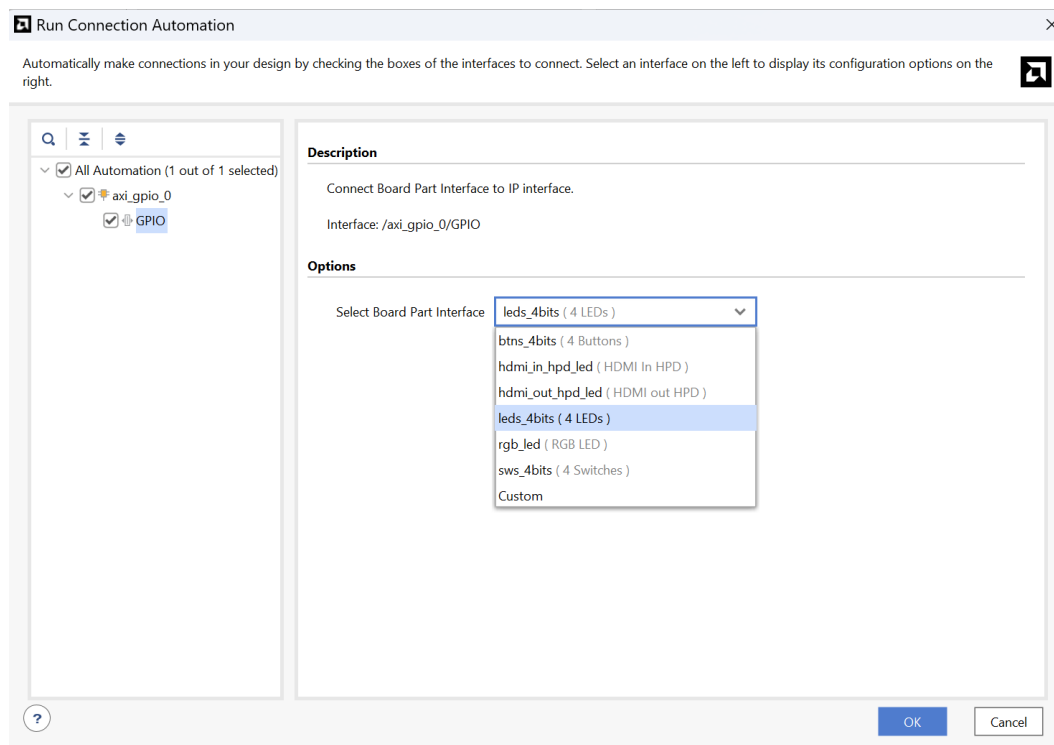


Hình 10: Tự động kết nối AXI



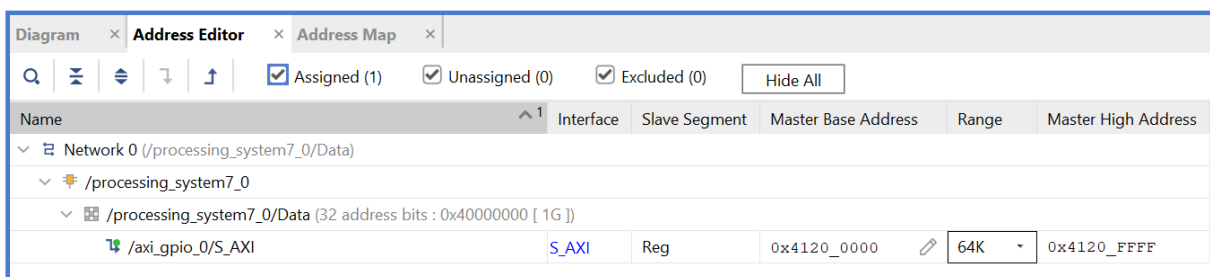
Hình 11: Sau khi kết nối AXI

- Click **Run Connection Automation** lần nữa:
- + Chọn `/axi_gpio_0/GPIO`
 - + Ở dropdown "Select Board Part Interface" → chọn **leds_4bits** (ZyBo có 4 LED — khác với ZedBoard là leds_8bits)
 - + **OK**

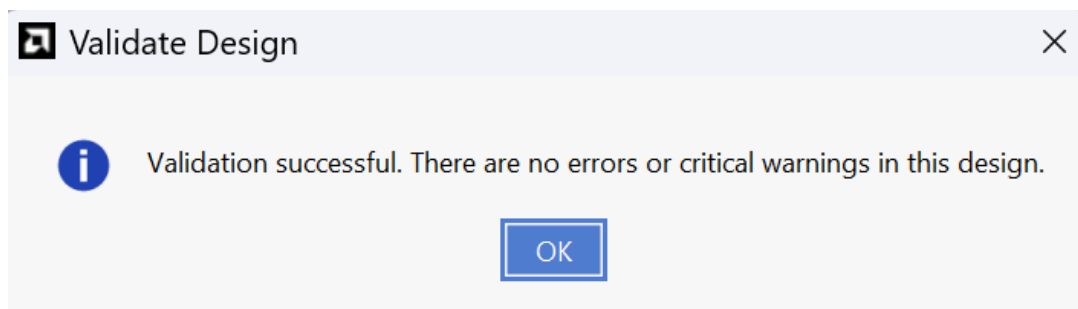


Hình 12: Chọn ngõ ra khỏi AXI là led 4 bit

- Click tab **Address Editor** ở phía trên canvas → mở rộng Data, kiểm tra AXI GPIO đã được map với range 64K.
- **File** → **Save Block Design** (Ctrl+S).
- **Tools** → **Validate Design** (hoặc nút check ở toolbar). Phải báo "validation successful" → **OK**.



Hình 13: Kiểm tra thanh ghi

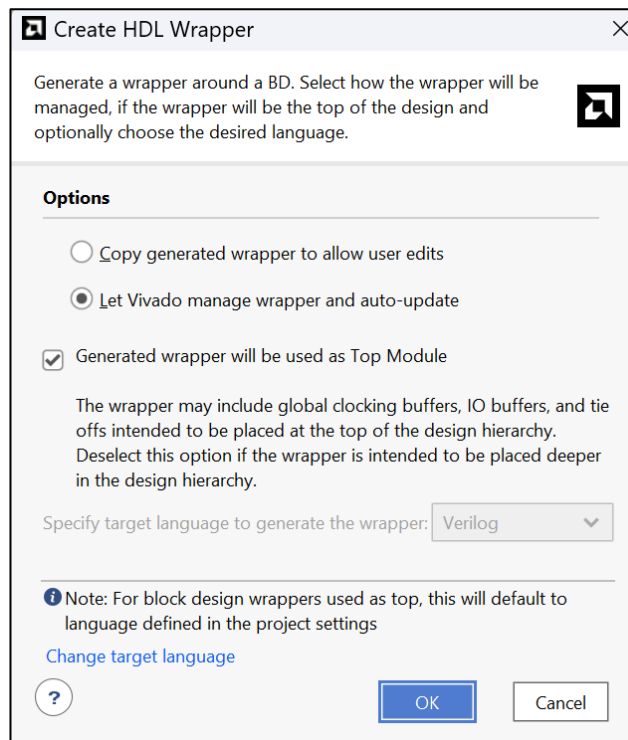


Hình 14: Kiểm tra thành công

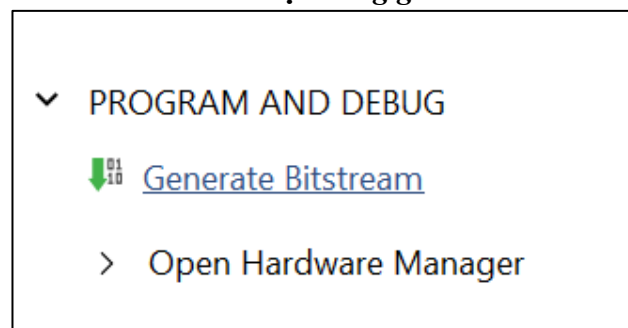
- Mở tab **Sources** (Window → Sources).
- Right-click `first_zynq_system` → **Create HDL Wrapper** → chọn **Let Vivado manage wrapper and auto-update** → **OK**.
- Flow Navigator → **Program and Debug** → **Generate Bitstream**.
- Nếu hỏi launch Synthesis + Implementation → **Yes**. Quá trình này chạy synth → implementation → bitstream (mất vài phút).
- Khi xong → chọn **Open Implemented Design** → **OK**.
- **File** → **Export** → **Export Hardware...**
- Tick **Include bitstream** → **OK**.

Lưu ý: phải có Implemented Design đang active thì option "Include bitstream" mới enable được (đó là lý do bước trc ta mở Implemented Design).

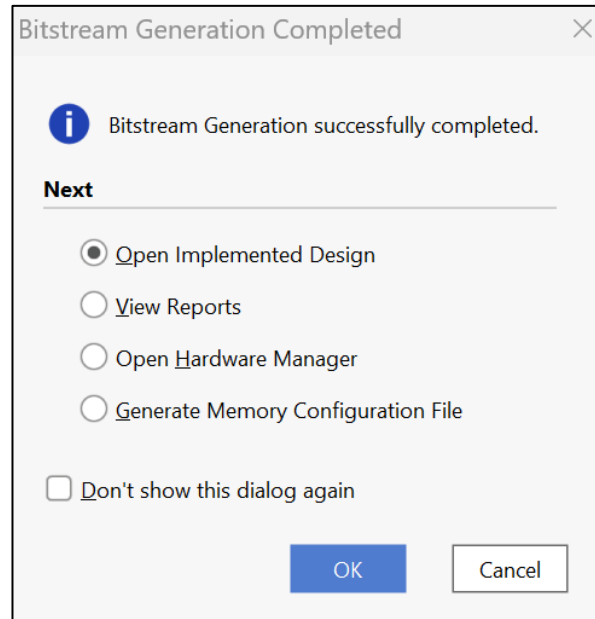
- **File** → **Launch SDK** → **OK**. SDK (Eclipse-based) sẽ mở ra với Hardware Platform Project có sẵn.



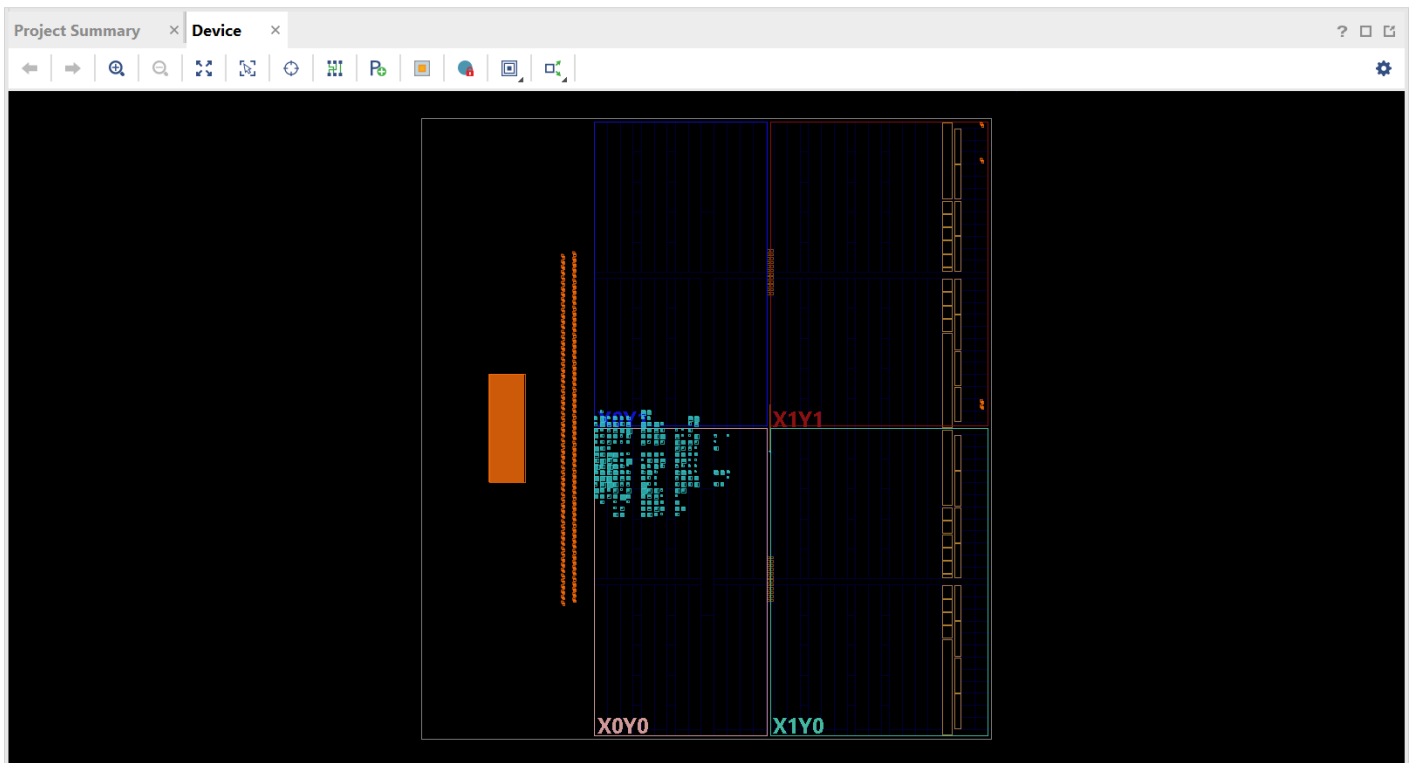
Hình 15: Tạo đóng gói HDL



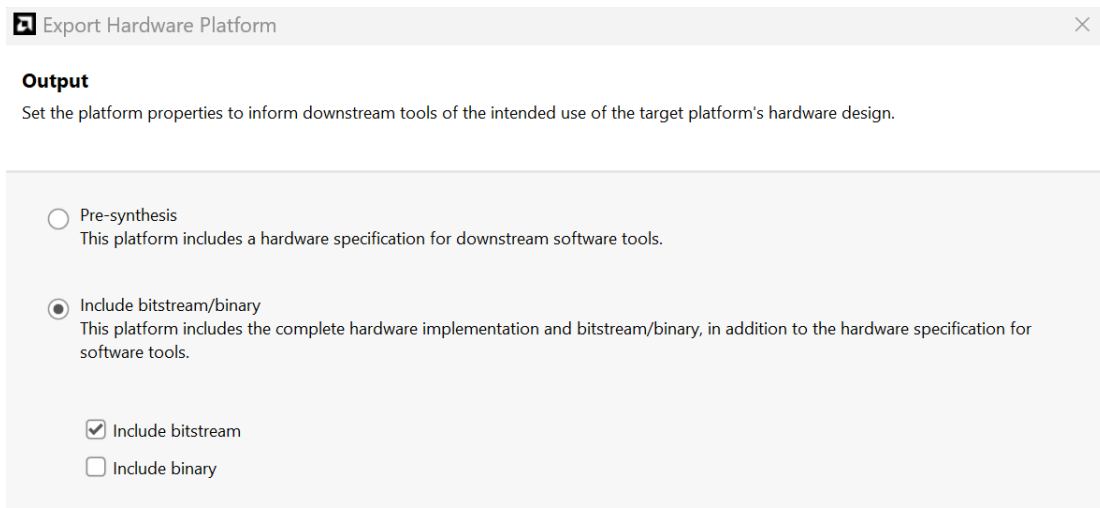
Hình 16: Tạo bitstream



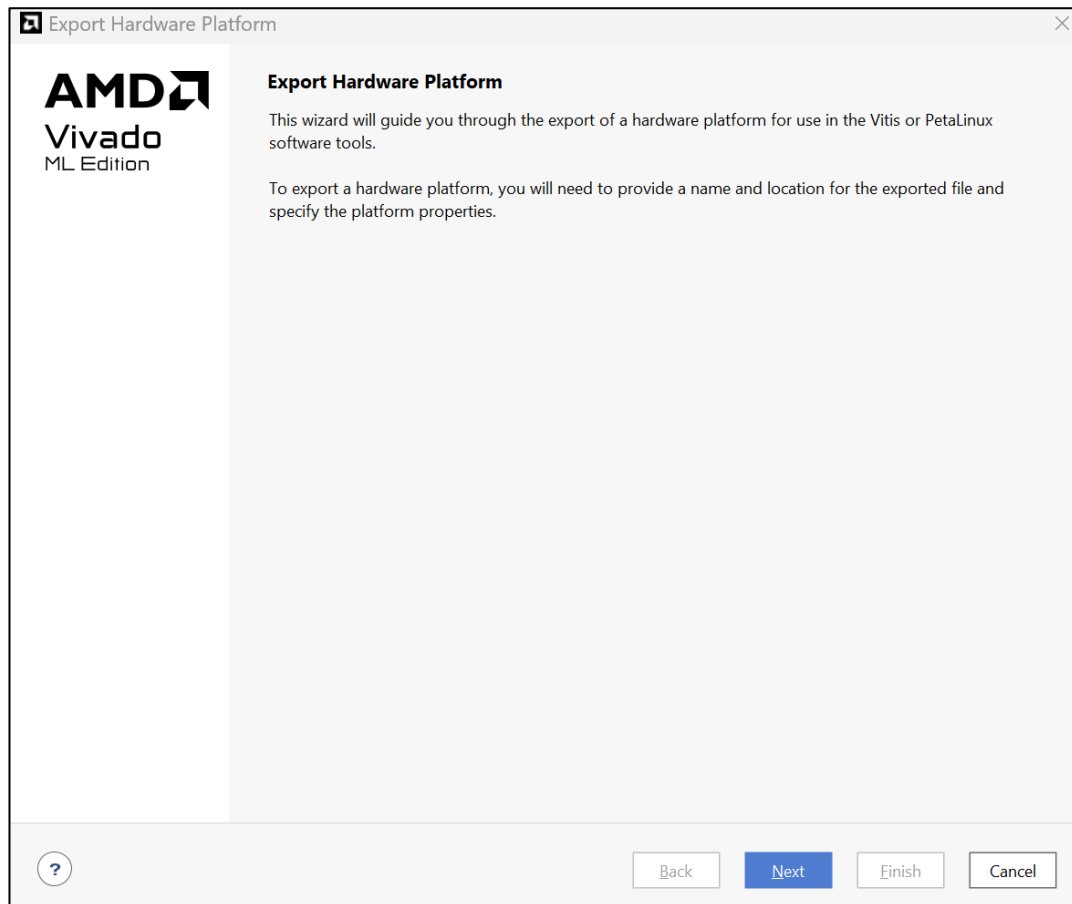
Hình 17: chọn open Implemented design



Hình 18: giao diện Device

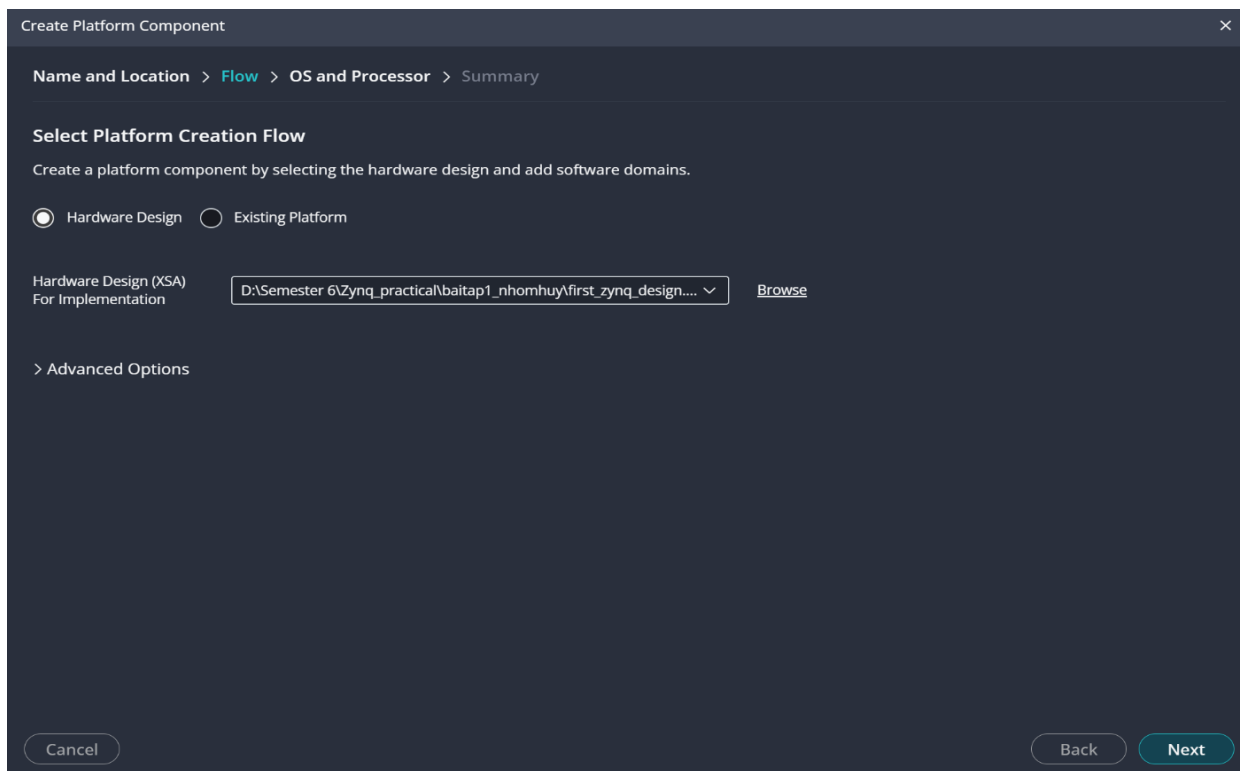


Hình 19: Xuất thành file (XSA)

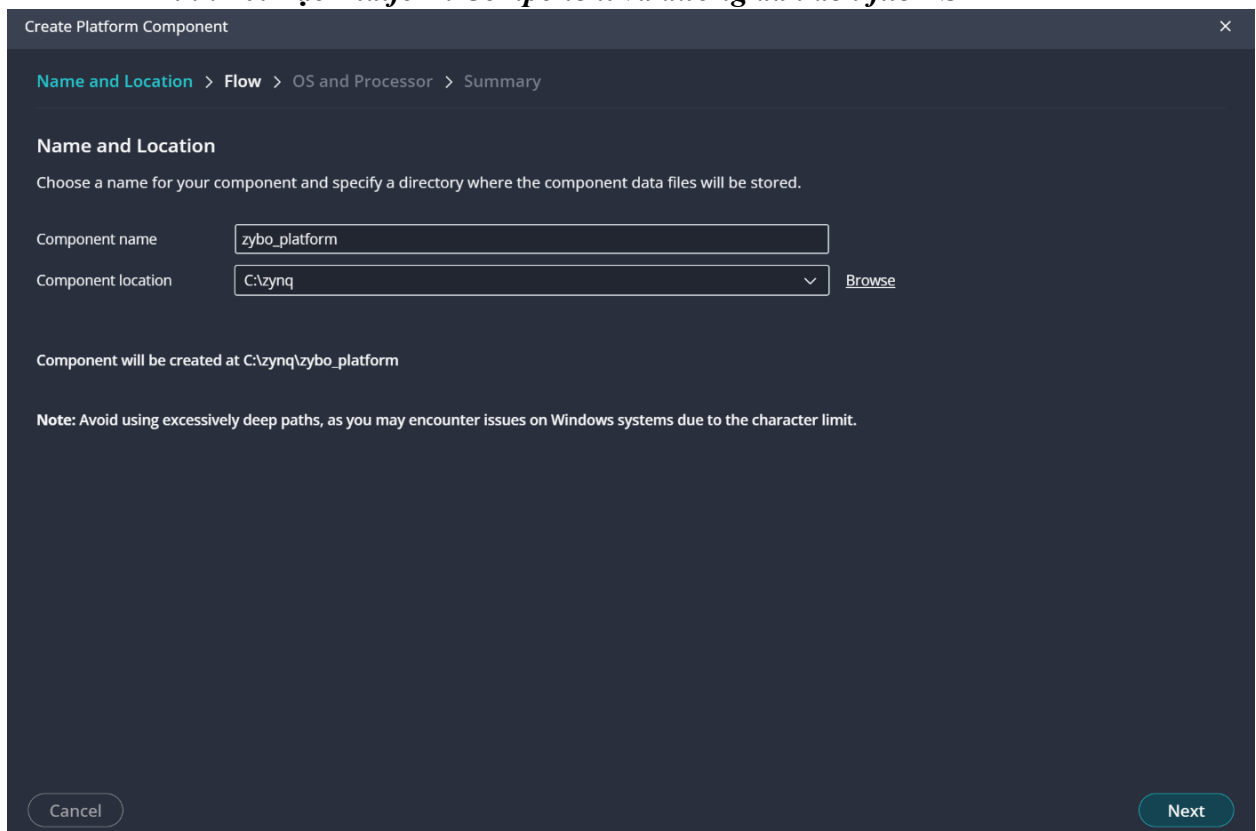


Hình 19: Xuất thành file (XSA)

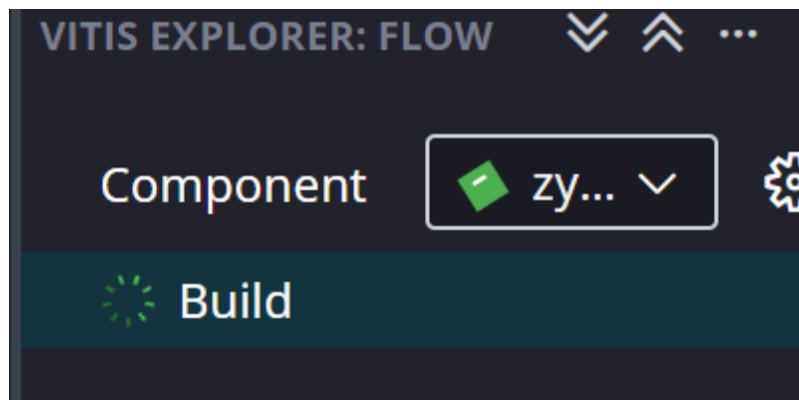
- Trước khi tạo App, phải build Platform Component (vì App phụ thuộc BSP của Platform):
1. Trong panel **Components** (bên trái) → click `zybo_platform`
 2. Panel **Flow** (giữa hoặc phải) → click nút **Build** (hoặc Ctrl+B)
 3. Đợi 30s-2 phút. Output panel sẽ in log build BSP. Cuối log phải có:



Hình 20: Tạo Platform Component và đường dẫn đến file XSA



Hình 21: Đặt tên và chọn đường dẫn

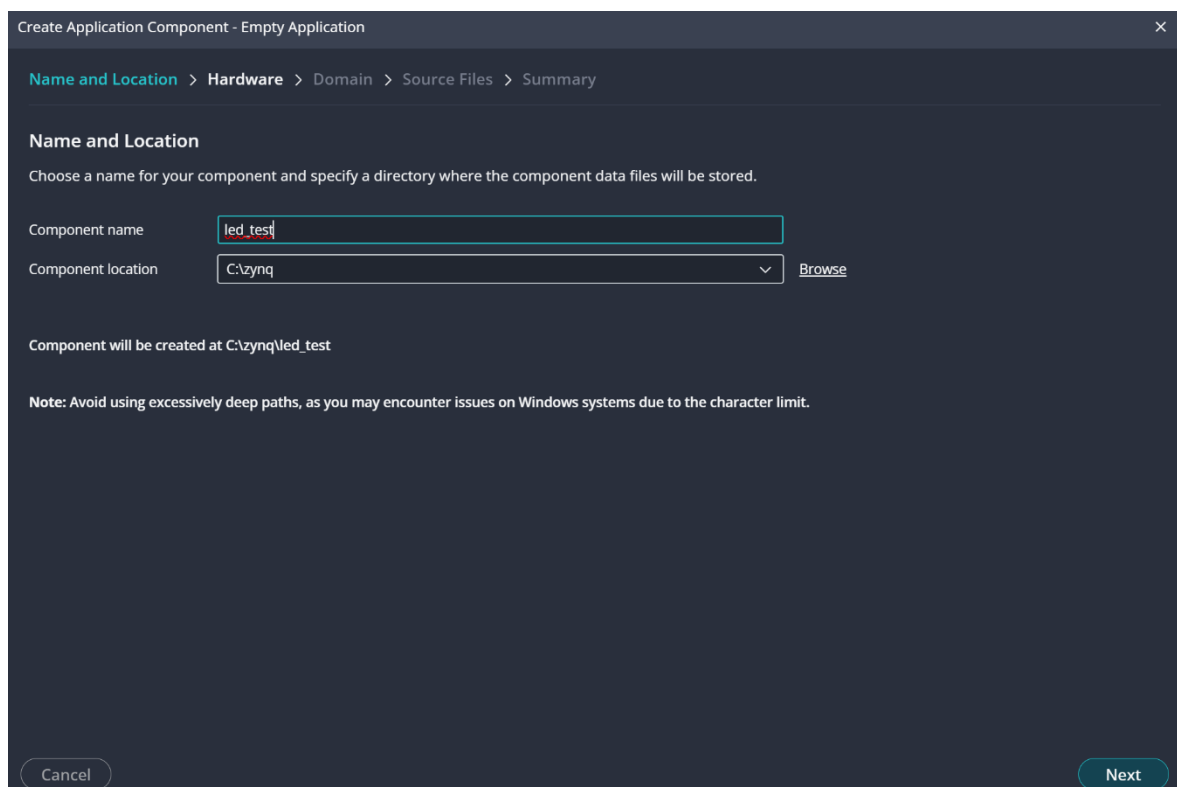


Hình 22: Build component

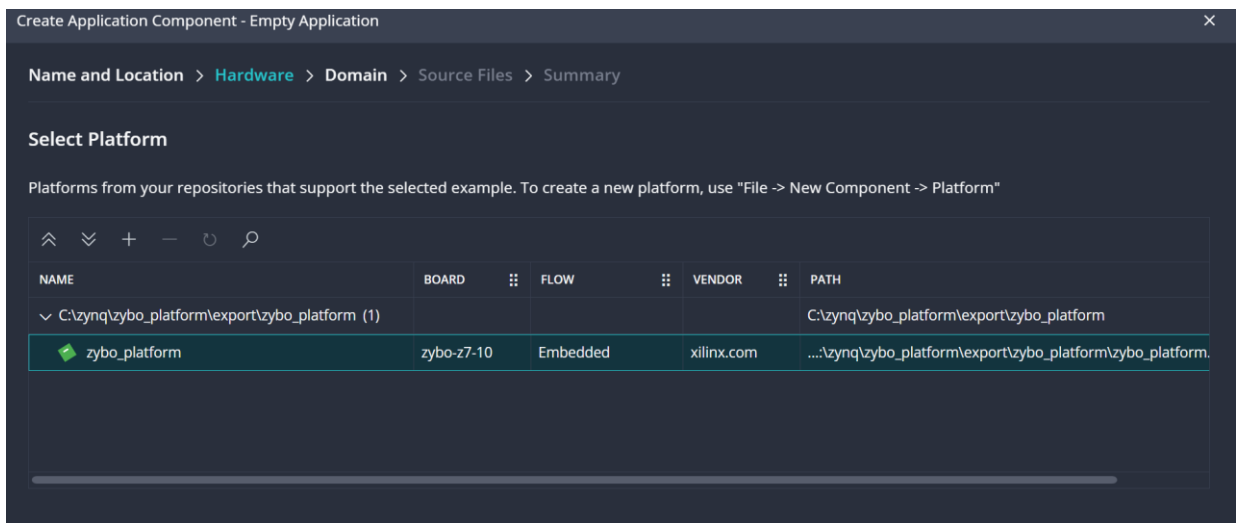
- **Tạo Application**

1. **File** → **New Component** → **Application...** (hoặc Welcome page → **Create Application Component**)
2. **Name:** LED_test → **Next**
3. **Platform:**
 - Chọn radio **Select a platform from repository**
 - Dropdown sẽ hiện zybo_platform vừa build → chọn nó → **Next**
4. **Domain:**
 - Dropdown sẽ tự hiện standalone_ps7_cortexa9_0 → giữ nguyên → **Next**
5. **Templates:**
 - Chọn **Empty Application (C)**
6. **Finish**

Application LED_test xuất hiện trong panel Components.

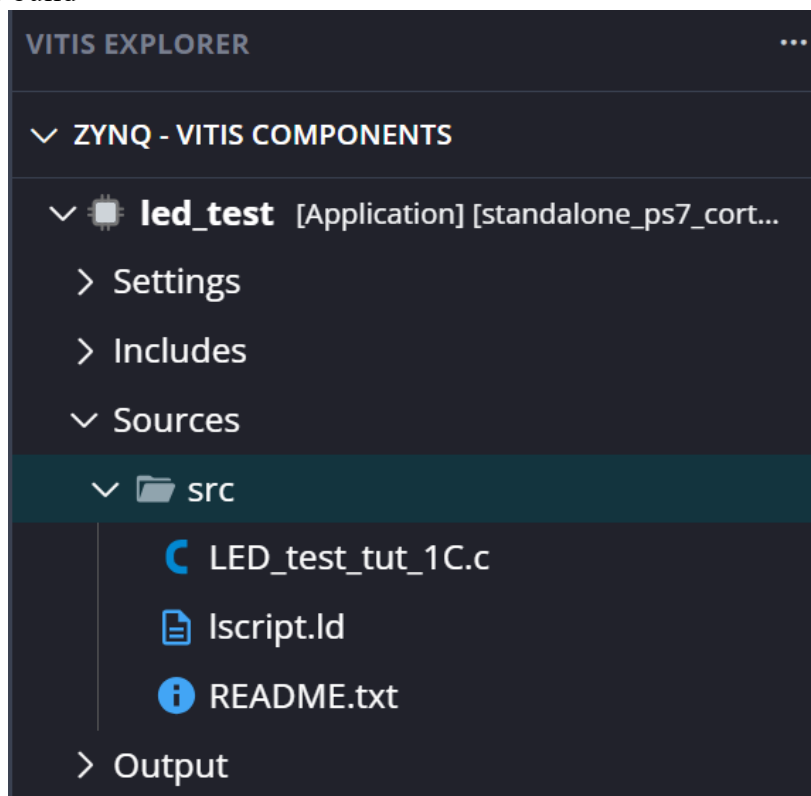


Hình 23: Tạo Application Component và đặt tên

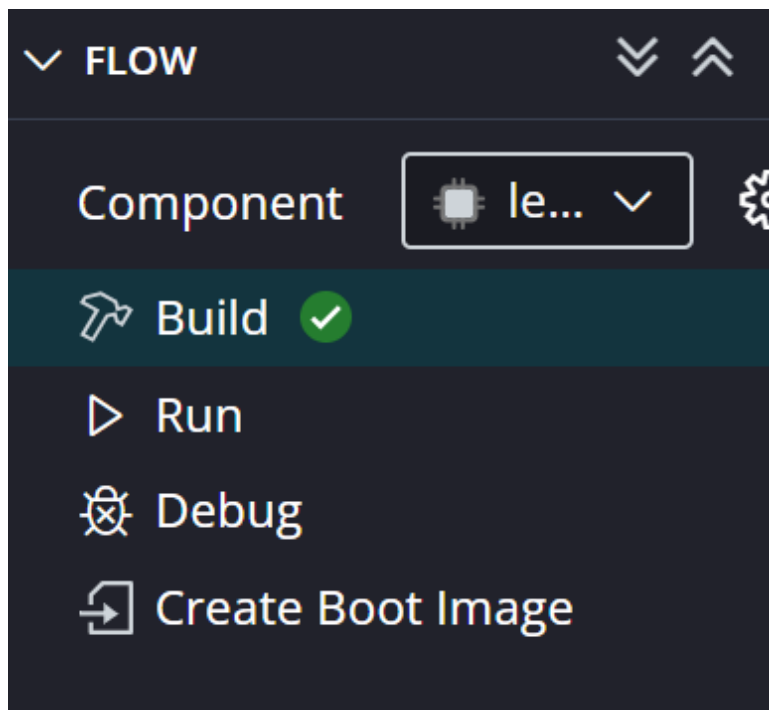


Hình 24: Chọn platform cho application

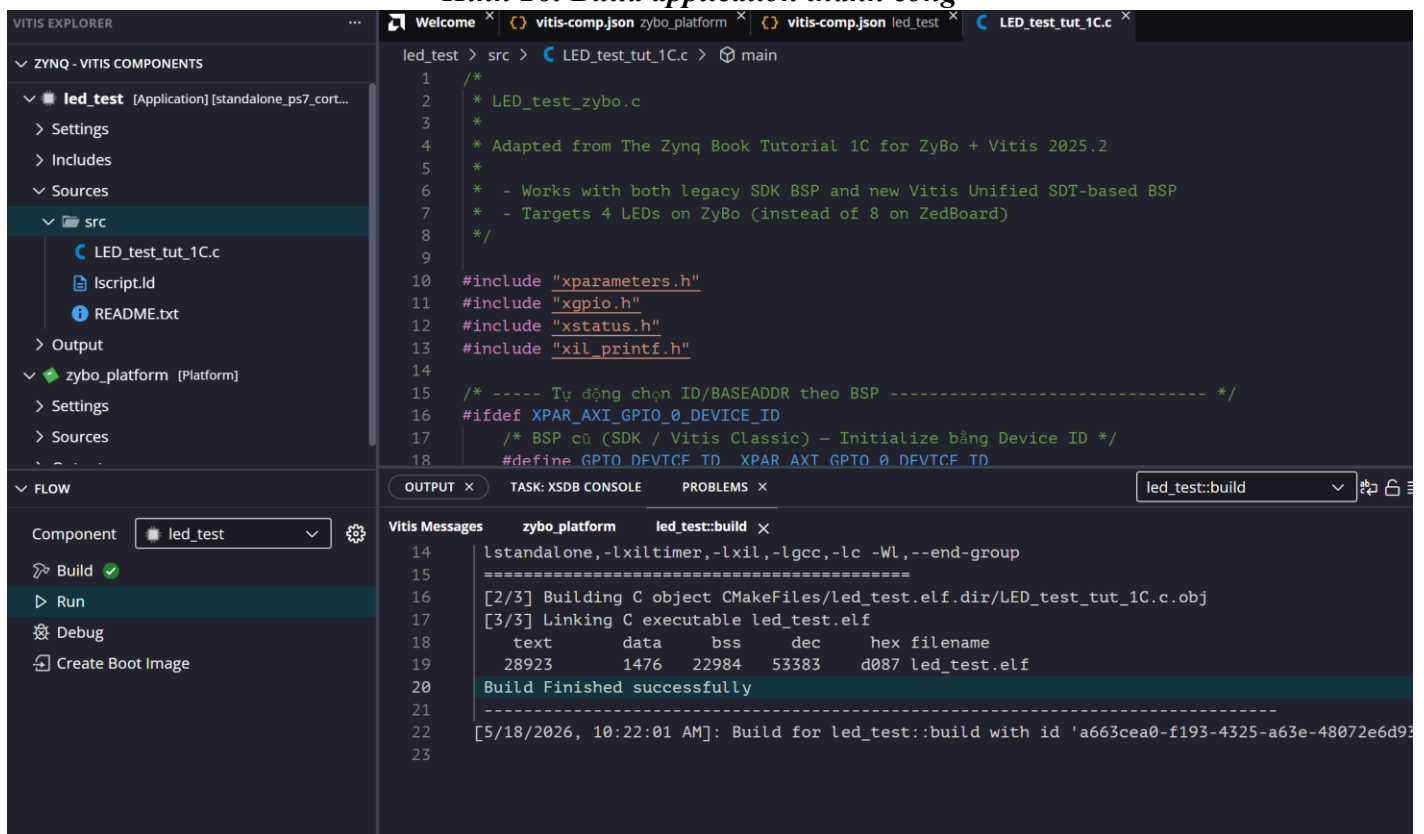
- Import file C để build



Hình 25: Import file test C vào source



Hình 26: Build application thành công



Hình 27: Giao diện gồm file, code, build

Link github: <https://github.com/lenhhuy/zynq-hw-sw-codesign.git>

Hình nhóm:

